if Babel.locale$_m apped$ == $nil then Babel.locale_mapped = true Babel.line breaking.add_before(Babel.locale_map, 1) Babel.loc_to_scr = Babel.chr_to_loc = Babel.chr_to_loc_cend Babel.locale_props[1].letters = false if Babel.script_blocks['Latn'] then Babel.loc_to_scr[1] = Babel.script_blocks['Latn'] Babel.locale_props[1].lg = 0 end if Babel.script_b$

Module 1: Algorithm Analysis & Amortization

Core Concepts

Worst-Case Analysis (Lec 2) Measures the maximum number of operations an algorithm will perform for any input of size n. This is the standard for analysis in this course.

Amortized Analysis (Lec 9, CLRS 16) Analyzes a *sequence* of operations to find the average cost per operation in the worst case. It is **not** the same as average-case analysis.

Asymptotic Notations (Lec 3)

For a given time function $T(n) \geq 0$:

$O(f(n))$ **(Big-Oh)** (Upper Bound) $\exists$ positive constants c, n_0 such that $T(n) \leq c \cdot f(n)$ for all $n \geq n_0$.

$\Omega(f(n))$ **(Big-Omega)** (Lower Bound) $\exists$ positive constants c, n_0 such that $T(n) \geq c \cdot f(n)$ for all $n \geq n_0$.

$\Theta(f(n))$ **(Big-Theta)** (Tight Bound) $\exists$ positive constants c_1, c_2, n_0 s.t. $c_1 f(n) \leq T(n) \leq c_2 f(n)$ for all $n \geq n_0$. (This means $T(n)$ is $O(f(n))$ and $\Omega(f(n))$).

Common Growth Rates (Lec 3)

$O(1) \subset O(\log n) \subset O(n) \subset O(n \log n) \subset O(n^2) \subset O(n^k) \subset O(2^n) \subset O(n!)$

Amortized Analysis Methods (Lec 9, CLRS 16)

1. Aggregate Analysis (Lec 9)

Show that for a sequence of n operations, the total cost $T(n)$ is $O(f(n))$. The amortized cost per operation is then $T(n)/n$.

- **Binary Counter (n increments):** Total cost = $\sum_{i=0}^{\log n} \frac{n}{2^i} \approx 2n = O(n)$. Amortized cost = $\mathbf{O(1)}$.
- **Dynamic Array (Push):** Total cost for n pushes = $O(n)$ (for n pushes) + $O(n)$ (for all copying). Amortized cost = $\mathbf{O(1)}$.

2. Accounting Method (CLRS 16.2)

Assign different *amortized costs* ($\hat{c}$) to operations.

- If $\hat{c} > c$ (actual cost), the extra charge is stored as **credit** on the data structure.
- If $\hat{c} < c$, the deficit is paid by stored credit.
- **Rule:** The total credit must **never be negative**.
- **Dynamic Array (Push):**
  - Let **Amortized Cost** $\hat{c} = 3$.
  - **Case 1: No resize (actual cost $c = 1$).** We pay 3. 1 pays for the push, 2 are stored as credit.
  - **Case 2: Resize (actual cost $c = n + 1$).** Occurs when $n-1$ items are present. There must be $(n-1)$ items, each with 2 units of credit, for a total of $2(n-1)$ credit. We use n of this credit to pay for the copy, and 1 to pay for the push.
  - We use $n+1$ credit, but have $2n-2$. Credit remains non-negative. Amortized cost is $\mathbf{O(1)}$.
  - $\hat{c}_i = c_i + \Delta\Phi = 1 + 2 = 3$.
  - **Case 2: Resize ($c_i = n$).** $\text{num}_i = n$, $\text{size}_i = 2(n-1)$. $\text{num}_{i-1} = n-1$, $\text{size}_{i-1} = n-1$.
  - $\Delta\Phi = (2n - 2(n-1)) - (2(n-1) - (n-1)) = 2 - (n-1) = 3 - n$.
  - $\hat{c}_i = c_i + \Delta\Phi = n + (3-n) = 3$.
  - In both cases, the amortized cost is $\mathbf{O(1)}$.

Module 2: Linear Data Structures (Lists, Stacks, Queues)

Arrays vs. Linked Lists (Lec 4, 5, 6)

A fundamental trade-off in data structures.

Operation	Array	Linked List
Read (by index)	$O(1)$	$O(n)$
Search	$O(n)$	$O(n)$
Insert (at front)	$O(n)$	$O(1)$
Insert (at end)	$O(1)$*	$O(1)$**
Delete (at front)	$O(n)$	$O(1)$
Delete (at end)	$O(1)$	$O(n)$***

*Amortized $O(1)$ with dynamic array (Lec 9). **Requires a `tail` pointer.
***Requires traversal to find predecessor, or a Doubly Linked List.

- **Sentinel Nodes (CLRS 10.2):** To simplify code, use a special dummy node 'L.nil' for the 'head' and to mark the 'next' of the tail. This removes edge cases for empty lists or single-node lists.

Stack ADT (LIFO - Last-In, First-Out) (Lec 6, 7)

Push(e) Insert element at the top. $O(1)$*.
Pop() Remove and return element from top. $O(1)$.
Top() Return top element without removing. $O(1)$.

*Amortized $O(1)$ using a dynamic array (Lec 9).

Application: Balanced Parentheses (Lec 7)

Algorithm to check if a string of brackets is balanced.

- Create an empty stack.
- Iterate through the string:
  - If char is an **opening** bracket ('(', '[', '{'), **push** it onto the stack.
  - If char is a **closing** bracket (')', ']', '}'):
    - If stack is empty → **return false**.
    - **Pop** the top element.
    - If popped bracket does not match the closing bracket → **return false**.
- After the loop, **return true** if the stack is empty, else **false**.

Complexity: $O(n)$ time, $O(n)$ space.

Queue ADT (FIFO - First-In, First-Out) (Lec 8)

Enqueue(e) Insert element at the back. $O(1)$*.
Dequeue() Remove and return element from front. $O(1)$**.
Front() Return front element without removing. $O(1)$.

*Amortized $O(1)$ using a dynamic/doubling array. **Requires a **circular array** implementation to avoid an $O(n)$ shift.

Application: Maze Solving / BFS (Lec 10)

- Queues are the core data structure for **Breadth-First Search (BFS)**.
- BFS explores "layer by layer", which is used to find the **shortest path** in unweighted graphs (like a maze).
- **Proof of Correctness:** This is a non-trivial proof by induction. See the proof in **Module 7** (Graphs).

General Tree Representation (CLRS 10.3)

How to store a tree where nodes have an *arbitrary* number of children?

- **Bad Idea:** Array of pointers at each node (wastes space).
- **Good Idea: Left-Child, Right-Sibling Representation.**
- Each node x has only two pointers:
  1. x.left_child: points to its first child.
  2. x.right_sibling: points to its next sibling.
- This creates a binary tree, where all "left" pointers go down one level, and all "right" pointers stay on the same level.

Module 3: Trees, Heaps, & Priority Queues

Tree ADT (Lec 12, 13)

A data structure that stores elements hierarchically.

Root The single top-most node.
Leaf (External) A node with no children.
Internal Node A node with one or more children.
Depth of v Length of the path from root to v. 'depth(root) = 0'.
Height of v Length of the longest path from v to a leaf. 'height(leaf) = 0'.
Height of Tree Height of the root.

Binary Trees (Lec 13, 14)

A tree where each node has at most two children: a **left** child and a **right** child.

- **Property:** A binary tree of height h has n nodes, where $h + 1 \leq n \leq 2^{h+1} - 1$.
- This implies the height h is at least $\Omega(\log n)$ and at most $O(n)$.

Tree Traversals (Lec 13, 14)

Systematic ways to visit all nodes in a tree.

Preorder (Root, Left, Right). Used to copy a tree.
Inorder (Left, Root, Right). Used in BSTs to get a sorted list.
Postorder (Left, Right, Root). Used to delete a tree or calculate directory sizes (Lec 13).
Level-Order (BFS). Visits nodes layer by layer using a **Queue**.

Binary Heap (Min-Heap) (Lec 14-16, CLRS 6)

A **Priority Queue** is often implemented using a Min-Heap.

Structure Property It is an **Almost Complete Binary Tree (ACBT)**. All levels are full except possibly the last, which is filled from left to right.
Height An ACBT with n nodes has height $h = \lfloor \log n \rfloor$. (CLRS 6.1)
Order Property (Min-Heap) For every node u (except the root), key(parent(u)) $\leq$ key(u). The minimum element is always at the root.

Array-Based Representation (Lec 15)

An ACBT can be stored efficiently in an array. For a node at 0-based index i:

- parent(i) = $\lfloor (i-1)/2 \rfloor$
- left(i) = $2i + 1$
- right(i) = $2i + 2$

Heap Operations (Lec 15, 16, CLRS 6.2-6.3)

Operation	Algorithm	Time
`GetMin`	Read root	$O(1)$
`Insert(k)`	Add to end, **Trickle-Up**	$O(\log n)$
`DeleteMin`	Replace root with last, **Trickle-Down (Heapify)**	$O(\log n)$
`BuildHeap`	Call 'Heapify' from $n/2$ down to 0	$\mathbf{O(n)}$

Correctness of Heap Operations

- **Trickle-Up (Insert):** After inserting k, the only violation is if $k <$ parent(k). Swapping k with its parent fixes this violation and moves the *only* potential violation one level up. This repeats until $k \geq$ parent(k) or k is the root.
- **Heapify(A, i) (Trickle-Down, CLRS 6.2):**
  - **Assumption:** The subtrees at 'left(i)' and 'right(i)' are already valid heaps.
  - **Goal:** Make the tree at i a valid heap.
  - **Logic:** Find the smallest of $\{i, \text{left}(i), \text{right}(i)\}$. If i is not the smallest, swap $A[i]$ with A[smallest]. This fixes the violation at i.
  - **Proof:** The swap *might* violate the heap property at the 'smallest' child's new position. By recursively calling 'Heapify(A, smallest)', we correct this. This continues down the tree until a leaf is reached.
  - **Time:** $O(h) = O(\log n)$.
- **Build-Heap(A) (Lec 16, CLRS 6.3):**
  - **Algorithm:** Call 'Heapify(i)' for $i = \lfloor n/2 \rfloor$ down to 0.
  - **Correctness (Loop Invariant):** "At the start of each iteration i, nodes $i+1, \ldots, n-1$ are all roots of valid max-heaps."
  - **Init:** True. All nodes from $\lfloor n/2 \rfloor + 1$ to $n-1$ are leaves, which are trivial heaps.
  - **Maintenance:** 'Heapify(i)' assumes children at $2i+1$ and $2i+2$ are heap roots. Since $2i+1 > i$, the invariant holds. 'Heapify' makes i a heap root, maintaining the invariant for the next (smaller) i.
  - **Termination:** When $i = 0$, node 0 is made a heap root. Since $1 \ldots n-1$ are already heap roots, the entire tree is a valid heap.
  - **Time Analysis:** $\mathbf{O(n)}$, not $O(n \log n)$. Most nodes are near the bottom and have small height. $\sum_{h=0}^{\log n} (n/2^{h+1}) \cdot O(h) = O(n)$.

Module 4: Heap Apps & Greedy Algorithms

Heap Sort (Lec 17, CLRS 6.4)

An in-place sorting algorithm with $O(n \log n)$ worst-case time.

Algorithm (for ascending order)

1. **Build Max-Heap:** Convert the input array A of n elements into a max-heap. ($O(n)$)
2. **Iterate** $i = (n-1)$ **down to 1:**
   - **Swap** root ($A[0]$, the max element) with $A[i]$.
   - **Shrink** the heap size by 1 (max element is now sorted at $A[i]$).
   - **Heapify(A, 0)** on the new root to restore the max-heap property on the $A[0 \dots i-1]$ subarray. ($O(\log n)$)

Total Time: $O(n)$ (Build) $+ (n-1) \times O(\log n)$ (Heapify) = $\mathbf{O(n \log n)}$.

Proof of Correctness

Loop Invariant: At the start of each iteration i:

1. The subarray $A[i+1 \dots n-1]$ contains the $(n-i)$ largest elements of A, *in sorted order*.
2. The subarray $A[0 \dots i]$ contains the $i+1$ smallest elements and is a valid max-heap.

- **Init:** $i = n-1$. The invariant is trivially true (subarray $A[n \dots n-1]$ is empty). 'Build-Heap' made $A[0 \dots n-1]$ a heap.
- **Maintenance:** Swapping $A[0]$ (the max) with $A[i]$ puts the next largest element in its correct sorted spot. 'Heapify(0)' restores the heap property for $A[0 \dots i-1]$.
- **Termination:** When $i = 0$, the loop terminates. The array $A[1 \dots n-1]$ contains the $n-1$ largest elements in sorted order. $A[0]$ must be the smallest.

Greedy Algorithm: Huffman Coding (Lec 17-20, CLRS 15.3)

A **greedy algorithm** that generates an optimal prefix-free binary code for data compression.

Problem Given symbols with frequencies p_i, find a binary code for each symbol to minimize the average encoding length: $\min \sum p_i \cdot$ (depth of symbol i).

Prefix-Free Codes No code is a prefix of another (e.g., if '01' is a code, '011' cannot be).

Tree Representation Prefix-free codes correspond to **full binary trees** where symbols are **only at the leaves**. The code for a symbol is the path from the root (0=left, 1=right).

Huffman's Algorithm (Greedy Strategy)

Builds the optimal tree from the bottom up.

1. **Initialize:** Create a **min-priority queue** (Min-Heap) of n "trees", where each tree is a single leaf node for a symbol, and its priority is its frequency.
2. **Iterate** $n-1$ **times:**
   - `DeleteMin` twice to get the two trees (T_x, T_y) with the **lowest frequencies** (p_x, p_y).
   - **Merge** them: Create a new parent node with T_x and T_y as children.
   - The new tree's frequency is $p_x + p_y$.
   - **Insert** this new, merged tree back into the min-priority queue.
3. **Finish:** The last tree remaining in the queue is the optimal Huffman tree.

Running Time: $O(n \log n)$, because it involves $O(n)$ 'Insert' and $O(n)$ 'DeleteMin' operations on a priority queue of size $O(n)$.

Proof of Correctness (CLRS 15.3)

Uses induction on $n = |\Sigma|$. Proves two key properties:

Greedy Choice Property In *some* optimal tree, the two lowest-frequency symbols (x, y) are sibling leaves at the maximum depth.

- *Proof (by exchange):* Take any optimal tree T^*. Let b, c be two siblings at max depth. Let x, y be the two lowest-frequency symbols. If $\{b, c\} \neq \{x, y\}$, we know $p_x \leq p_b$ and $p_y \leq p_c$. Swap x with b and y with c. The new cost can only decrease or stay the same. Thus, an optimal tree with x, y as deep siblings exists.

Optimal Substructure Let T' be an optimal tree for the subproblem where x, y are replaced by a new symbol z with $p_z = p_x + p_y$. Let T be the tree formed by replacing z in T' with a node that has children x, y.

- We have $L(T) = L(T') + p_x + p_y$.
- The cost of T is the cost of T' plus a *fixed constant* $(p_x + p_y)$.
- Therefore, minimizing $L(T)$ is equivalent to minimizing $L(T')$.

Conclusion The algorithm makes the (safe) greedy choice and then optimally solves the resulting subproblem (by induction). Thus, the final tree T is optimal.

Module 5: Hash Tables

Core Idea & Operations

The main goal is to support 'Insert', 'Delete', and 'Lookup' (Search) in average-case $\mathbf{O(1)}$ time.

- **Setup:** We have a large **Universe** U of possible keys. We want to store a (much smaller) dynamic set $S \subseteq U$.
- **Structure:** An array A (buckets) of size n, where $n \approx |S|$.
- **Hash Function:** A function $h : U \to \{0, \dots, n-1\}$ that maps keys to array indices (buckets).

Applications

De-duplication (Lec 21) Check if a new item in a stream has been seen before.

- For new item x, 'Lookup(x)'. If not found, 'Insert(x)'.
- $\mathbf{O(1)}$ per item.

2-SUM Problem (Lec 21) Given array A (size n) and target t, find if $x, y \in A$ s.t. $x + y = t$.

- 'Insert' all elements of A into a hash table H. ($O(n)$)
- For each $x \in A$, 'Lookup(t-x)' in H. ($O(n) \times O(1) = O(n)$)
- **Total Time:** $\mathbf{O(n)}$. (Beats $O(n \log n)$ sorting approach).

Collisions (Lec 21, 23)

Collisions are inevitable (by Pigeonhole Principle) when $|U| > n$. The **Birthday Paradox** shows they are far more likely than expected (e.g., $n = 10k$, 400 objects $\rightarrow$ 99.9% collision chance).

Collision Resolution Strategies (Lec 22)

1. Chaining (Separate Chaining)

- Each bucket $A[i]$ holds a pointer to a **linked list** of all elements that hash to i.
- **Load Factor:** $\alpha = |S|/n$. Can be > 1.
- **Operations:**
  - 'Insert(x)': Insert x at the head of list $A[h(x)]$. **Time:** $O(1)$.
  - 'Lookup(x)': Search list $A[h(x)]$.
  - 'Delete(x)': Search and delete from list $A[h(x)]$.
- **Performance:** All ops depend on list length. If h is good ("spreads data"), average list length is α.
- **Expected Time:** $\mathbf{O(1 + \alpha)}$. If we maintain $n = \Theta(|S|)$, then $\alpha = O(1)$, and all ops are $O(1)$.

2. Open Addressing

- At most one object per bucket. Requires $n \geq |S|$ (so $\alpha \leq 1$).
- **Probing:** If $h(x)$ is full, probe for the next empty slot using a probe sequence.
- **Linear Probing:** Try $h(x), (h(x) + 1)\%n, (h(x) + 2)\%n, \ldots$
- **Double Hashing:** Try $h_1(x), (h_1(x) + h_2(x))\%n, (h_1(x) + 2h_2(x))\%n, \ldots$
- **Delete:** Tricky. Cannot just empty the slot. Must mark it as "DELETED".
- **Performance:** Heavily degrades as $\alpha \rightarrow 1$. Needs $\alpha \ll 1$.

Good Hash Functions (Lec 23)

Goal "Spreads out the data" evenly, mapping similar keys to different buckets.

Bad Functions `h(x) = first 3 digits` (bad for phone numbers) or `h(x) = x mod n` where $n = 2^k$ (bad for data with common low-bit patterns).

Method `h(x) = (hash_code(x)) mod n`

- **Hash Code:** Map key x (e.g., string) to a large integer.
- **Compression:** Map the integer to $\{0, \ldots, n-1\}$.

Choosing n (bucket count) n should be a **prime number** that is not too close to a power of 2. This helps break up patterns in the input data.

Universal Hashing To defeat a "pathological dataset" (adversary), we use a *family* of hash functions $\mathcal{H}$ and pick one $h \in \mathcal{H}$ at random. This guarantees good average performance, no matter the input data.

Module 6: Search Trees (BST, AVL, B-Trees)

Binary Search Tree (BST) (Lec 24, 25, CLRS 12)

A binary tree that maintains a sorted order.

The BST Invariant

For any node x in the tree:

- All keys in x's **left** subtree are $\leq$ key(x).
- All keys in x's **right** subtree are $\geq$ key(x).

Property (CLRS Thm 12.1): An `InorderTraversal` of a BST outputs its elements in sorted order. *Proof (by induction):* A subtree of 1 node is sorted. Assume Inorder(T_L) and Inorder(T_R) are sorted. Then Inorder(T) = [Inorder(T_L)] + [root] + [Inorder(T_R)]. By the BST invariant, all items in $T_L \leq$ root $\leq$ all items in T_R. Thus, the full list is sorted.

Operations (all $O(h)$, where h is tree height)

`Search(k)` Start at root. If $k <$ key(x), go left. If $k >$ key(x), go right. If $x =$ NULL, k is not found.

`Minimum(x)` Start at x, follow `left` child pointers until NULL.

`Maximum(x)` Start at x, follow `right` child pointers until NULL.

`Insert(z)` `Search` for key k. The (NULL) leaf position where the search ends is where z is inserted.

Theorem 12.4 (CLRS): The expected height of a **randomly built** BST on n distinct keys is $\mathbf{O}(\log \mathbf{n})$. This means that on average, all operations are efficient.

Successor(x) (Lec 25, CLRS 12.2)

Finds the node with the smallest key that is *larger* than key(x).

- **Case 1: x has a right subtree.** The successor is the `Minimum()` node in x's right subtree.
- **Case 2: x has NO right subtree.** Go up the tree using `parent` pointers. The successor is the **first ancestor** y such that x is in the **left subtree** of y. (This is the first "right turn" you take on the path up). If you reach the root and no such y exists, x was the maximum.

Delete(z) (Lec 25, CLRS 12.3)

This is the most complex operation, formalized by CLRS with a helper 'TRANSPLANT(u, v)' which replaces subtree u with subtree v.

- **Case 1: z has no left child.** Replace z with its right child (which could be NULL).
- **Case 2: z has no right child.** Replace z with its left child.
- **Case 3: z has two children.**
  1. Find z's **successor**, $y =$ `Minimum`(z.right).
  2. y is guaranteed to be in z's right subtree and will have **no left child**.
  3. **If y is z's direct child:** Replace z with y, and y adopts z's left child.
  4. **If y is not z's direct child:**
     - Replace y with its own right child (a Case 1 op).
     - y adopts z's right child.
     - Replace z with y, and y adopts z's left child.

Problem: All operations are $O(h)$. If keys are inserted in sorted order, the tree becomes a linked list with $h = O(n)$. Operations degrade to $O(n)$.

Augmented BSTs (Lec 26)

Idea: Store extra data in each node to answer new types of queries.

- **Rule:** The augmented data must be computable from the node's key and its children's augmented data in $O(1)$ time.
- **Example: Subtree Size.** Store $size(x)$ at each node.
- **Update Rule:** $size(x) = size(\text{left}(x)) + size(\text{right}(x)) + 1$.
- **New Operation:** `Rank(t)` (How many nodes are $\leq t$?) in $O(h)$.

AVL Trees (Self-Balancing BST) (Lec 27, 28, 29)

Goal: Guarantee $h = O(\log n)$ by keeping the tree balanced.

AVL Property (Height-Balance) For *every* node x, the heights of its children can differ by at most 1: $|\text{height}(\text{left}(x)) - \text{height}(\text{right}(x))| \leq 1$.

Complexity All BST operations are guaranteed $\mathbf{O}(\log \mathbf{n})$.

Insert & Delete in AVL Trees

1. Perform the standard BST 'Insert' or 'Delete'.
2. Walk back up from the modified node to the root, updating the 'height' of each ancestor.
3. Find the **first (lowest) ancestor** z that violates the AVL property.
4. Perform the correct **Rotation** operation at z to restore balance.
5. *Proof of Correctness (Insert):* 'Insert' requires at most 1 (single or double) rotation. The rotation at z is designed such that the **new height** of the rotated subtree is the **same as the height of z before the insertion**. This means no ancestors above z become unbalanced.
6. *Delete:* May require $O(\log n)$ rotations as you travel up, as a rotation at one node might unbalance its parent.

Rotations (The Fix)

Let z be the unbalanced node, y be its taller child, and x be y's taller child.

Case 1: Left-Left (Single **Right Rotation** at z)

Case 2: Right-Right (Single **Left Rotation** at z)

Case 3: Left-Right (**Left Rotation** at y, then **Right Rotation** at z)

Case 4: Right-Left (**Right Rotation** at y, then **Left Rotation** at z)

(2,4) Trees (Lec 30, 31, GTM 10.4)

A balanced M-way tree ($M = 4$) where nodes can hold multiple keys.

Size Property Every internal node has 2, 3, or 4 children (and 1, 2, or 3 keys).

Depth Property All leaves are at the **same level**.

Height $h = \mathbf{O}(\log \mathbf{n})$ (proven by min 2 children/node $\rightarrow$ $h \leq \log_2 n$).

Insert in (2,4)-Tree

1. `Search` to find the correct leaf node to insert k.
2. **If leaf has ¡ 3 keys:** Add k to the node. Done.
3. **If leaf is full (3 keys):** This is an **Overflow**.
   (a) Add k to the full node (temporarily a "5-node" with 4 keys).
   (b) **Split** the node: Promote the middle key (k_2) up to the parent.
   (c) The remaining keys form two new nodes (with k_1 and k_3, k_4) which are attached to the parent.
   (d) If the parent is now full, **propagate the split upwards**.
   (e) If the root splits, the tree height grows by 1.

Correctness: The split operation is the only structural change, and it perfectly maintains the (2,4) properties (size and depth).

Delete (Bottom-Up) (GTM 10.4.3)

1. `Search` for key k. If k is not in a leaf, swap it with its successor (which must be in a leaf).
2. Remove k from the leaf node.
3. **If leaf is now empty (0 keys):** This is an **Underflow**.
   - **Case 1: Transfer.** If an adjacent sibling has ≥ 2 keys, perform a "rotation". Move a key from the parent down to the empty node, and move a key from the sibling up to the parent.
   - **Case 2: Fusion.** If adjacent siblings are minimal (only 1 key), **merge** the empty node, its minimal sibling, and the key from the parent.
   - This fusion may cause the parent to underflow, so the process may **propagate upwards**. If the root underflows and is empty, it is deleted and the height shrinks by 1.

Delete (Top-Down / Pre-emptive)

Idea: Ensures we never delete from a minimal node (1-key node), avoiding upward propagation.

1. **Handle Non-Leaf:** If k is in an internal node, swap it with its inorder successor (which is in a leaf). The problem is now to delete the successor from a leaf.
2. **Pre-emptive Traversal:** Start at the root. As we traverse *down* to the target leaf, at each node x we visit:
3. **If x is minimal (1 key):**
   - **Case 1 (Transfer):** If an adjacent sibling has ≥ 2 keys, perform a transfer (rotation) through the parent.
   - **Case 2 (Fusion):** If adjacent siblings are also minimal, perform a fusion with a sibling and the parent's separating key. (If the root is part of a fusion and becomes empty, the fused child becomes the new root.)
4. Now x is guaranteed to have ≥ 2 keys.
5. **Descend:** Move to the appropriate child (which may have changed after a fusion).
6. **Final Deletion:** When we reach the target leaf, it is *guaranteed* to have ≥ 2 keys. Remove k. No underflow occurs.

B-Trees (Lec 32, CLRS 18)

The generalization of (2,4)-trees, ideal for disk-based memory.

Properties (min-degree t)

- Each node holds k keys, where $t - 1 \leq k \leq 2t - 1$.
- Each internal node with k keys has $k + 1$ children.
- All leaves are at the same depth.
- Root has at least 1 key (or 0 if tree is empty).

Height $h = O(\log_t n)$. This is excellent for disk, as t can be large (e.g., 1000), making h very small.

Search in B-Tree (CLRS 18.2)

$O(t)$ work per node, but $O(1)$ disk access. Total disk accesses: $\mathbf{O(\log_t n)}$.

Insert in B-Tree (CLRS 18.3)

Idea: Avoids the "underflow-propagation" of (2,4) trees by being **pre-emptive**. We never insert into a full node.

1. Start at root. As we traverse *down* to find the leaf:
2. **If we encounter a full node** x **(**$2t-1$ **keys):**
3. **Split** x **first** using 'B-TREE-SPLIT-CHILD'.
4. 'B-TREE-SPLIT-CHILD(parent, i, x)':
   - Splits x (the i-th child of parent) into two nodes, x and z.
   - x gets the first $t-1$ keys. z gets the last $t-1$ keys.
   - The median key of x (key t) moves up into the parent.
5. After splitting, we continue our search from the (now non-full) parent.
6. **Finally:** We reach the target leaf, which is *guaranteed* not to be full. Insert key k.

Correctness: This maintains all B-Tree properties. The only way height grows is if the root itself splits.

Module 7: Graph Algorithms

Graph Definitions (Lec 33)

Graph $G = (V, E)$. $n = |V|, m = |E|$.
Path Sequence of vertices $(v_1, \ldots, v_k)$ where $(v_i, v_{i+1}) \in E$.
Cycle A path that starts and ends at the same vertex. A graph with no cycles is **acyclic**. A **DAG** is a Directed Acyclic Graph.
Handshake Lemma $\sum_{v \in V} \deg(v) = 2m$.

Graph Representations (Lec 34)

Adjacency List Array of n linked lists. $Adj[u]$ stores neighbors of u.
- **Memory:** $\mathbf{O(n+m)}$. **Standard for sparse graphs.**
- **Check Edge** (u, v)**:** $O(\deg(u))$.

Adjacency Matrix $n \times n$ matrix A where $A[u][v] = 1$ if $(u, v) \in E$.
- **Memory:** $\mathbf{O(n^2)}$. **Standard for dense graphs.**
- **Check Edge** (u, v)**:** $O(1)$.

Breadth-First Search (BFS) (Lec 35, CLRS 20.2)

Idea: Explores in "layers," 1 edge away, then 2, etc.

- **Data Structure:** A **FIFO Queue**.
- **Algorithm (from** s**):**
  1. Mark s as explored (GRAY), 'dist(s)=0', 'enqueue(s)'.
  2. While queue is not empty:
  3. $u \leftarrow$ `dequeue()`.
  4. For each neighbor v of u:
  5. If v is unexplored (WHITE):
  6. Mark v GRAY, 'dist(v) = dist(u) + 1', 'parent(v)=u', 'enqueue(v)'.
  7. Mark u as finished (BLACK).
- **Complexity:** $\mathbf{O(n+m)}$.

Application: Shortest Paths (Unweighted).

- **Correctness (White Path Theorem):** BFS finds the shortest path $d(s, v)$ to all reachable v.
- **Proof (by Induction on distance** k**):** (Lec 11/12)
- **Hypothesis** $Hyp(k)$**:** BFS finds all nodes at distance k (set S_k) after finding all nodes at distance $k-1$ (set S_{k-1}), and 'dist(v)=k' $\forall v \in S_k$.
- **Base** $k = 1$**:** $S_0 = \{s\}$ is enqueued. Its neighbors (S_1) are enqueued next, all with 'dist = 0+1 = 1'. True.
- **Step** $Hyp(k) \to Hyp(k+1)$**:** The queue ensures all nodes from S_k are processed before any from S_{k+1}. When a $u \in S_k$ is dequeued, it discovers all its unvisited neighbors v. These v must be in S_{k+1} (if they were in S_k or earlier, they'd be visited). BFS sets 'dist(v) = dist(u) + 1 = k+1'. True.

Depth-First Search (DFS) (Lec 37, CLRS 20.3)

Idea: Explores as far as possible before backtracking.

- **Data Structure:** A **LIFO Stack** (or recursion).
- **Algorithm (recursive, from** s**):**
  1. Mark s as explored (GRAY) and set discovery time $d[s]$.
  2. For each neighbor v of s:
  3. If v is unexplored (WHITE):
  4. 'parent(v)=u', call 'DFS(v)'.
  5. Mark s as finished (BLACK) and set finish time $f[s]$.
- **Complexity:** $\mathbf{O(n+m)}$.

Properties (CLRS 20.3):

- **Parenthesis Theorem:** The discovery/finish intervals are nested. For any two nodes u, v, either their intervals are disjoint, or one is contained within the other (iff one is a descendant of the other in the DFS tree).
- **Edge Types:**
  - **Tree Edge:** To a WHITE child.
  - **Back Edge:** To a GRAY ancestor (indicates a **CYCLE**).
  - **Forward/Cross Edge:** To a BLACK node.

DFS Applications (Lec 38, 39, CLRS 20.4, 20.5)

Topological Sort (for DAGs) A linear ordering where for all edges (u, v), u appears before v.
- **Algorithm:** Run DFS. As each vertex finishes, insert it onto the **front** of a linked list.
- **Correctness:** For any edge (u, v), DFS must finish v before it finishes u (if it saw (u, v) and v was GRAY, it's a cycle). Thus, $f[v] < f[u]$. Since the list is sorted by decreasing $f[]$, u comes before v.

Strongly Connected Components (SCCs) (Kosaraju's)
1. Run DFS on G to get finish times $f[u]$.
2. Compute G^T (reverse all edges).
3. Run DFS on G^T, visiting vertices in order of **decreasing** $f[u]$.
4. Each tree in the 2nd DFS's forest is one SCC.

Correctness: (Lec 39) The 1st DFS finds a "source" SCC

(the one with the highest finish time u). The 2nd DFS (on G^T) starts at u and can only explore u's SCC, because all edges now point *into* that component from other SCCs, so the DFS gets "trapped" in the SCC.

Weighted Shortest Paths: Dijkstra (CLRS 22.3)

Finds shortest paths from s in a **weighted** graph with **no negative edges**.

- **Data Structure:** A **Min-Priority Queue** (Min-Heap).
- **Algorithm (from s):**
  1. Init: 'dist[v] = ∞' for all v, 'dist[s] = 0'.
  2. Add all v to Min-Heap Q (keyed by 'dist[v]').
  3. While Q is not empty:
  4. $u \leftarrow Q$.`DeleteMin()`.
  5. For each neighbor v of u:
  6. **Relaxation:** If 'dist[u] + weight(u,v) ¡ dist[v]':
  7. 'dist[v] = dist[u] + weight(u,v)'
  8. 'parent(v) = u'
  9. `DecreaseKey`(Q, v)
- **Complexity:** $O(n \log n + m \log n)$ (naive heap) or $\mathbf{O(n \log n + m)}$ (with Fibonacci heap or as $m \geq n$ $\mathbf{O(m \log n)}$ with binary heap).
- **Correctness (Cut Property):** When Dijkstra adds u to the set of "finished" nodes, 'dist[u]' is the true shortest path. (Proof: Assume not. Let u be the first node added with the wrong distance. Consider the true shortest path $s \dots x \to y \dots u$, where x is finished and y is not. Because x is finished, $d(s, y) \leq d(s, u)$. And since $d(s, y) < d(s, u)$ (non-neg edges), the algorithm would have chosen y before u. Contradiction.)

Special Case: 0-1 BFS : If all weights are 0 or 1, use a **Deque**. When relaxing an edge: if weight is 0, 'push_front(v)'; if weight is 1, 'push_back(v)'. This maintains the "layers" and achieves $O(n + m)$.

Minimum Spanning Tree (MST) (Lec 40, CLRS 21)

Finds a min-cost spanning tree in a connected, undirected, weighted graph.

- **Generic Correctness (Cut Property):** For any cut (partition of V into $S, V - S$), if (u, v) is the **unique** minimum-weight edge crossing the cut, it **must** be in the MST. (Proof: Assume not. Adding (u, v) to the MST T^* creates a cycle. This cycle must have another edge (x, y) crossing the cut. $c(x, y) > c(u, v)$. Swapping (u, v) for (x, y) gives a cheaper tree. Contradiction.)

Kruskal's Algorithm (Lec 40, 41, CLRS 21.2)

Idea: A greedy algorithm that builds the MST by adding the cheapest edges that don't form a cycle.

1. **Sort** all m edges by weight.
2. Init: $T = \emptyset$. Init a **DSU** with n sets (one per vertex).
3. For each edge (u, v) in sorted order:
4. If `Find`$(u) \neq$ `Find`(v):
5. $T = T \cup \{(u, v)\}$
6. `Union`(u, v)

Time: $\mathbf{O(m \log m)}$ or $\mathbf{O(m \log n)}$ (dominated by edge sort).

Prim's Algorithm (CLRS 21.2)

Idea: A greedy algorithm that "grows" the MST from a single vertex s.

1. **Data Structure:** A **Min-Priority Queue** (Min-Heap) Q.
2. Init: For all v, 'key[v] = ∞', 'parent[v] = NULL'. Set 'key[s] = 0'.
3. Add all v to Q (keyed by 'key[v]').
4. While Q is not empty:
5. $u \leftarrow Q$.`DeleteMin()`.
6. For each neighbor v of u:
7. If $v \in Q$ and weight$(u, v) <$ key$[v]$:
8. 'parent[v] = u', 'key[v] = weight(u,v)'
9. `DecreaseKey`(Q, v)

Time: $\mathbf{O(m \log n)}$ (with a binary heap).

Disjoint Set Union (DSU) (Lec 41, CLRS 19)

Data structure for maintaining disjoint sets. Used by Kruskal's.

`MakeSet(x)` Create a new set $\{x\}$.
`Find(x)` Return the representative (root) of x's set.
`Union(x, y)` Merge the sets containing x and y.

Optimizations (Required for good performance):

1. **Union-by-Size (or Rank):** In 'Union(x, y)', make the root of the **smaller** tree point to the root of the **larger** tree.
   - **Proof:** This guarantees tree height is $O(\log n)$. A node's depth only increases when its set is merged into a larger one, at least doubling the set size. This can only happen $O(\log n)$ times.
2. **Path Compression:** During 'Find(x)', make every node on the path from x to the root point directly to the root.

Complexity: With both optimizations, a sequence of m operations (on n sets) takes $\mathbf{O(m \cdot \alpha(n))}$ time, where $\alpha(n)$ is the inverse Ackermann function, which is practically $O(1)$ (grows *extremely* slowly, e.g., $\alpha(n) \leq 4$ for any practical n).

Module 8 (Part 1): Problems (Pre-BST & DP)

Module 1/2: Arrays, Search, Stacks, & Lists

Problem: 2-Sum (Lec 21, Blind 75)

- **Problem:** Given an array of integers nums and a target, return indices of two numbers such that they add up to target. (Also called "Check for Pair with Given Sum" in GFG).
- **Solution Idea:** Use a Hash Map (Module 5). Iterate through the array once. For each element x, check if target - x is already in the map. If yes, return the indices. If no, add x and its index to the map.
- **Complexity:** $O(n)$ time, $O(n)$ space.

Problem: 3Sum (Blind 75)

- **Problem:** Find all unique triplets in nums that sum to zero.
- **Solution Idea:** Sort the array first ($O(n \log n)$). Then, iterate through the array with a main pointer i. For each i, use two new pointers j = i+1 and k = n-1. Move j and

k toward each other, checking the sum. If sum ¡ 0, move j right. If sum ¿ 0, move k left.
- **Key:** Must skip duplicate elements for i, j, and k to ensure unique triplets.
- **Complexity:** $O(n^2)$ time, $O(\log n)$ or $O(n)$ space for sorting.

Problem: Search in Rotated Sorted Array (Blind 75)

- **Problem:** Given a sorted array rotated at some pivot, search for a target in $O(\log n)$ time.
- **Solution Idea:** Use a modified Binary Search (Module 1). In each step, check if the mid element is in the "left" sorted portion or the "right" sorted portion.
- 1. Find mid. If nums[mid] == target, return.
  2. **If left half is sorted** (nums[low] ¡= nums[mid]):
     - If target is in this half (nums[low] ¡= target ¡ nums[mid]), search left (high = mid - 1).
     - Otherwise, search right (low = mid + 1).
  3. **If right half is sorted** (nums[mid] ¡ nums[high]):
     - If target is in this half (nums[mid] ¡ target ¡= nums[high]), search right (low = mid + 1).
     - Otherwise, search left (high = mid - 1).
- **Complexity:** $O(\log n)$ time, $O(1)$ space.

Problem: Find Majority Element (from GFG Arrays.pdf)

- **Problem:** Find the element that appears more than $n/2$ times.
- **Solution Idea (Boyer-Moore):** Maintain a candidate and a count. Iterate the array. If count == 0, set candidate = nums[i]. If nums[i] == candidate, count++. Else, count–.
- **Key:** A second pass is needed to verify if the candidate truly appears ¿ $n/2$ times, (though not if one is guaranteed to exist).
- **Complexity:** $O(n)$ time, $O(1)$ space.

Problem: K-th Smallest of Two Sorted Arrays (Hard)

- **Problem:** Given two sorted arrays A (size n) and B (size m), find the k-th smallest element in their union.
- **Solution Idea:** Use Binary Search on the *smaller* array (assume A, $n \leq m$) to find a partition i.
- We binary search for i in the range $[\max(0, k - m), \min(k, n)]$.
- Let $j = k - i$. We have found the correct partition when: $A[i-1] \leq B[j]$ AND $B[j-1] \leq A[i]$.
- (Handle edge cases $i = 0, i = n, j = 0, j = m$ with $-\infty$ and $+\infty$).
- When found, the answer is $\max(A[i-1], B[j-1])$.
- **Complexity:** $\mathbf{O}(\log(\min(\mathbf{n}, \mathbf{m})))$ time, $O(1)$ space.

Problem: Container With Most Water (Blind 75)

- **Problem:** Find two lines in height array which, with the x-axis, form a container holding the most water.
- **Solution Idea:** Use a "two-pointer" approach. Start with l=0 and r=n-1. Calculate the area. To maximize area, you must move the pointer corresponding to the **shorter** line.
- **Complexity:** $O(n)$ time, $O(1)$ space.

Problem: Trapping Rain Water (Blind 75, GFG Arrays.pdf)

- **Problem:** Given an elevation map, compute how much water it can trap.
- **Solution Idea (Optimal):** Use two pointers l=0 and r=n-1, and two variables max_l and max_r. If max_l ¡ max_r, process the left pointer l (water is max_l - height[l]) and move l right. Otherwise, process and move r left.
- **Complexity:** $O(n)$ time, $O(1)$ space.

Problem: Product of Array Except Self (Blind 75, GFG Arrays.pdf)

- **Problem:** Given nums, return answer where answer[i] is the product of all elements of nums except nums[i]. Must be $O(n)$ and no division.
- **Solution Idea:** Use two passes. (1) Left-to-right pass to store prefix products in answer. (2) Right-to-left pass, multiplying by a running suffix product.
- **Complexity:** $O(n)$ time, $O(1)$ space (if output array doesn't count).

Problem: Count Inversions (from GFG Divide-And-Conquer.pdf)

- **Problem:** Find the number of pairs (i, j) such that $i < j$ and $A[i] > A[j]$.
- **Solution Idea:** Use a modified Merge Sort. During the merge step, when an element from the **right** subarray (R[j]) is moved before an element from the **left** subarray (L[i]), it means R[j] is smaller than all remaining elements in the left subarray.
- if L[i] ¡= R[j]: move L[i].
- else: move R[j], inversions += (mid - i + 1).
- **Complexity:** $O(n \log n)$ time (same as Merge Sort).

Problem: Set Matrix Zeroes (LeetCode)

- **Problem:** If an element in an $M \times N$ matrix is 0, set its entire row and column to 0. Do this in-place.
- **Solution Idea:** Use the first row and first column of the matrix itself as storage to mark rows/cols for zeroing. Use two boolean flags to handle whether the first row/col themselves originally contained a zero.
- **Complexity:** $O(M \cdot N)$ time, $O(1)$ space.

Problem: Valid Parentheses (Lec 7, Blind 75, GFG Stack.pdf)

- **Problem:** Check if a string of '(', ')', '[', ']', '', '' is valid.
- **Solution Idea:** This is the canonical Stack application. Iterate the string. Push opening brackets. When a closing bracket is found, pop and check for a match. If stack is empty on pop, or no match, or stack not empty at end, return false.
- **Complexity:** $O(n)$ time, $O(n)$ space.

Problem: Next Greater Element (from GFG Stack.pdf)

- **Problem:** For each element in an array, find the first element to its right that is greater than it.
- **Solution Idea:** Use a Stack. Iterate the array from **right to left**.
- For each element x:

- while stack is not empty and stack.top() ¡= x: pop() from stack.
- if stack is empty: result[i] = -1.
- else: result[i] = stack.top().
- push(x) onto the stack.
- **Complexity:** $O(n)$ time (each element is pushed/popped once), $O(n)$ space.

Problem: Largest Rectangular Area in Histogram (from GFG Stack.pdf)

- **Problem:** Find the largest rectangle of all-1s in a binary matrix (or largest area in a histogram).
- **Solution Idea:** (For Histogram) Use a stack to store indices. We want to find the nearest smaller bar on the *left* and *right* for each bar i.
- Iterate i from 0 to n: while stack not empty and hist[stack.top()] ¿= hist[i]:
- tp = stack.pop(). The hist[tp] bar's right bound is i, left bound is stack.top().
- area = hist[tp] * (i - stack.top() - 1). Update max_area.
- push(i) onto stack.
- **Complexity:** $O(n)$ time, $O(n)$ space.

Problem: Evaluate Reverse Polish Notation (LeetCode)

- **Problem:** Evaluate an arithmetic expression in Reverse Polish Notation (RPN).
- **Solution Idea:** Classic Stack problem. Iterate tokens. If number, push. If operator, pop two numbers, perform operation, push result.
- **Complexity:** $O(n)$ time, $O(n)$ space.

Problem: Linked List Cycle (Lec 37, Blind 75, GFG Linked-List.pdf)

- **Problem:** Determine if a linked list has a cycle.
- **Solution Idea:** Floyd's Tortoise and Hare. slow moves 1 step, fast moves 2. If they meet, a cycle exists.
- **Complexity:** $O(n)$ time, $O(1)$ space.

Problem: Linked List Cycle II (LeetCode)

- **Problem:** If a cycle exists, return the node where it begins.
- **Solution Idea:** Find the meeting point meet. Reset a new pointer start to head. Move start and meet one step at a time. Their new meeting point is the cycle's start.
- **Complexity:** $O(n)$ time, $O(1)$ space.

Problem: Reverse a Linked List (from GFG Linked-List.pdf)

- **Problem:** Reverse a singly linked list.
- **Solution Idea (Iterative):** Use three pointers: prev = NULL, curr = head, next = NULL.
- while (curr != NULL): next = curr.next, curr.next = prev, prev = curr, curr = next.
- Return prev (which is the new head).
- **Complexity:** $O(n)$ time, $O(1)$ space.

Problem: Merge Two Sorted Lists (from GFG Linked-List.pdf)

- **Problem:** Merge two sorted linked lists into one sorted list.
- **Solution Idea:** Create a dummy head node. Maintain a tail pointer. Compare l1.val and l2.val. Append the smaller node to tail, and advance that list's pointer.
- **Complexity:** $O(n+m)$ time, $O(1)$ space.

Problem: Intersection Point of Two Lists (from GFG Linked-List.pdf)

- **Problem:** Find the node at which two linked lists intersect.
- **Solution Idea 1 (Hash):** Add all nodes of l1 to a HashSet. Iterate l2 and return the first node found in the set. $O(n+m)$ time, $O(n)$ space.
- **Solution Idea 2 (Pointers):** Get lengths len1, len2. Move the pointer of the longer list by abs(len1-len2) steps. Now, move both pointers one step at a time. The node where they meet is the intersection. $O(n+m)$ time, $O(1)$ space.

Problem: Remove Nth Node From End of List (Blind 75)

- **Problem:** Remove the n-th node from the end of a list in one pass.
- **Solution Idea:** Use two pointers. Move fast n steps ahead. Then move fast and slow together until fast.next is NULL. slow is now at the node before the target.
- **Complexity:** $O(L)$ time (one pass), $O(1)$ space.

Problem: Implement Queue using Stacks (from GFG Queue.pdf)

- **Problem:** Implement Enqueue and Dequeue using two stacks.
- **Solution Idea (Amortized $O(1)$ Dequeue):** Use s1 (input) and s2 (output).
- **Enqueue(x):** s1.push(x). $O(1)$.
- **Dequeue():** if s2 is empty: while s1 is not empty: s2.push(s1.pop()).
- return s2.pop(). $O(1)$ amortized.
- **Complexity:** Amortized $O(1)$ for both operations.

Module 3/4: Heaps & Priority Queues

Problem: Find Median from Data Stream (Blind 75, GFG Heaps.pdf)

- **Problem:** Design a class to support addNum(int) and findMedian() from a stream.
- **Solution Idea:** Use two heaps: a **Max-Heap** left_half (smaller half) and a **Min-Heap** right_half (larger half). Rebalance after each add so sizes differ by at most 1.
- **Complexity:** addNum is $O(\log n)$, findMedian is $O(1)$.

Problem: Merge k Sorted Lists (Blind 75, GFG Heaps.pdf)

- **Problem:** Merge k sorted linked lists into one.
- **Solution Idea:** Use a **Min-Priority Queue** (Min-Heap) of size k. Insert the head of all k lists. While heap is not empty: DeleteMin node u, add u to result, Insert $u.next$ into heap.

- **Complexity:** $O(N \log k)$ time, where N is total elements. $O(k)$ space.

Problem: Top K Frequent Elements (Blind 75)

- **Problem:** Given nums and k, return the k most frequent elements.
- **Solution Idea (Heap):** (1) Build a Hash Map of (number, frequency) ($O(n)$). (2) Create a **Min-Heap** of size k. (3) Iterate the map, pushing (freq, num). If heap.size() ¿ k, pop().
- **Complexity:** $O(n \log k)$ time, $O(n + k)$ space.

Problem: Connect n Ropes with Minimum Cost (from GFG Heaps.pdf)

- **Problem:** Given n ropes of different lengths, connect them into one rope with minimum cost (cost of connecting a and b is $a + b$).
- **Solution Idea:** This is Huffman's idea (Lec 19). Use a **Min-Priority Queue**.
- Insert all n rope lengths into the heap.
- while heap.size() ¿ 1: a = heap.DeleteMin(), b = heap.DeleteMin(). new_rope = a + b. total_cost += new_rope. heap.Insert(new_rope).
- **Complexity:** $O(n \log n)$ time.

Module 5: Hash Tables

Problem: Longest Substring Without Repeating Characters (Blind 75)

- **Problem:** Find the length of the longest substring without repeating characters.
- **Solution Idea:** Use the "Sliding Window" technique with a Hash Set (or Hash Map char -¿ index). Maintain window $[l, r]$. Move r. If char[r] is a duplicate (in set or map), move l to the right past the first occurrence.
- **Complexity:** $O(n)$ time, $O(k)$ space (where k is size of charset).

Problem: LRU Cache (Hard, Blind 75, GFG Hashing.pdf)

- **Problem:** Design a cache with get and put in $O(1)$ time. It must evict the "Least Recently Used" item when full.
- **Solution Idea:** Combine a **Hash Map** (map¡key, Node*¿) for $O(1)$ lookup and a **Doubly Linked List (DLL)** to maintain recency order.
- **get(key):** Look up node in map. Move node to head of DLL.
- **put(key, val):** If key exists, update value, move node to head. If new: create node, add to head. If full: evict tail of DLL, remove tail.key from map.
- **Complexity:** $O(1)$ for both get and put.

Problem: Find Duplicates in O(n) time (from GFG Hashing.pdf)

- **Problem:** Given an array of n numbers (from 1 to n), find all duplicates.
- **Solution Idea (In-place Hash):** Iterate the array. For each element x, go to index idx = abs(x) - 1. If nums[idx] is negative, x is a duplicate. Otherwise, mark nums[idx] as negative (nums[idx] = -nums[idx]).
- **Complexity:** $O(n)$ time, $O(1)$ space.

Module X: Dynamic Programming (New Section)

Problem: Coin Change (Blind 75)

- **Problem:** Find the fewest number of coins to make a given amount, given coin denominations.
- **Solution Idea (Bottom-Up DP):** Create dp[amount+1] initialized to ∞. Set dp[0] = 0.
- for i from 1 to amount:
- for c in coins:
- if i - c ¿= 0:
- dp[i] = min(dp[i], 1 + dp[i-c])
- The answer is dp[amount].
- **Complexity:** $O(A \cdot C)$ time, $O(A)$ space, where A=amount, C=num coins.

Problem: Longest Increasing Subsequence (LIS) (Blind 75)

- **Problem:** Find the length of the longest strictly increasing subsequence.
- **Solution Idea 1 (DP):** dp[i] = length of LIS *ending at index i*. $O(n^2)$.
- **Solution Idea 2 (Patience Sorting):** Maintain an array tails storing the smallest ending element for all LIS of a given length. For each x in nums, find the first tails[i] ¿= x and replace it (using binary search). If no such element, append x.
- **Complexity 2:** $O(n \log n)$ time, $O(n)$ space.

Problem: Longest Common Subsequence (LCS) (Blind 75)

- **Problem:** Find the length of the LCS between text1 and text2.
- **Solution Idea (DP):** dp[i][j] = LCS of text1[0..i] and text2[0..j].
- if text1[i] == text2[j]: dp[i][j] = 1 + dp[i-1][j-1]
- else: dp[i][j] = max(dp[i-1][j], dp[i][j-1])
- **Complexity:** $O(n \cdot m)$ time, $O(n \cdot m)$ space.

Problem: Word Break (Blind 75)

- **Problem:** Can s be segmented into a sequence of dictionary words?
- **Solution Idea (DP):** dp[i] = boolean, true if s[0..i-1] can be broken.
- dp[0] = true. for i from 1 to n: for j from 0 to i-1:
- if dp[j] and s[j..i-1] in wordDict: dp[i] = true; break;
- **Complexity:** $O(n^2)$ time (due to substring check), $O(n)$ space.

Module 8 (Part 2): Problems (Post-BST)

Module 6: Trees, BSTs, Tries

Problem: Validate Binary Search Tree (Blind 75)

- **Problem:** Determine if a binary tree is a valid BST.
- **Solution Idea:** Use a recursive DFS. Pass down the valid range (min_val, max_val) allowed for the current node. The invariant must hold for all ancestors.
- isValid(node, min, max):
- if node.key ¡= min —— node.key ¿= max: return false.

- return isValid(node.left, min, node.key) && isValid(node.right, node.key, max).
- **Complexity:** $O(n)$ time, $O(h)$ space.

Problem: BST Deletion (from GFG Binary-Search-Tree.pdf)

- **Problem:** Delete a node z from a BST (Lec 25).
- **Solution Idea:**
  - **Case 1 (0 or 1 child):** Replace z with its child.
  - **Case 2 (2 children):** Find z's **inorder successor** y (the minimum node in z.right). y is guaranteed to have at most one (right) child.
  - Copy y.key into z.key.
  - Recursively delete y (which is now a Case 1 problem) from the right subtree.
- **Complexity:** $O(h)$ time.

Problem: Kth Smallest Element in a BST (Blind 75)

- **Problem:** Find the k-th smallest element in a BST.
- **Solution Idea 1 (Inorder):** Perform an inorder traversal. Stop at the k-th visited node. $O(n)$ time.
- **Solution Idea 2 (Augmented BST):** (Lec 26) If nodes store size(left_subtree), you can find it in $O(h)$ time.
- **Complexity 2:** $O(h)$ time.

Problem: Lowest Common Ancestor of a BST (from GFG)

- **Problem:** Find the LCA of two nodes p and q in a BST.
- **Solution Idea:** This is much simpler than a general BT. Start from the root.
- if p.key ¡ root.key && q.key ¡ root.key: recurse on root.left.
- if p.key ¿ root.key && q.key ¿ root.key: recurse on root.right.
- else: root is the LCA (the point where p and q split).
- **Complexity:** $O(h)$ time, $O(1)$ space (iterative).

Problem: Convert Sorted Array to Balanced BST (from GFG)

- **Problem:** Given a sorted array, create a height-balanced BST.
- **Solution Idea:** This is a Divide and Conquer approach.
  1. The **middle** element of the array arr[mid] becomes the root.
  2. The left subarray arr[0...mid-1] recursively forms the root.left subtree.
  3. The right subarray arr[mid+1...n-1] recursively forms the root.right subtree.
- **Complexity:** $O(n)$ time (visits each node once).

Problem: Lowest Common Ancestor of a Binary Tree (Blind 75)

- **Problem:** Find the LCA of two nodes p and q in a general binary tree.
- **Solution Idea:** Use a recursive DFS find(node, p, q).
- **Base Case:** if node == NULL or node == p or node == q, return node.
- **Recurse:** left = find(node.left), right = find(node.right).
- **Combine:** if left && right: return node (it's the LCA). else: return left or right (whichever is not NULL).
- **Complexity:** $O(n)$ time, $O(h)$ space.

Problem: Construct Tree from Preorder and Inorder (Blind 75)

- **Problem:** Given preorder and inorder traversal arrays, construct the binary tree.
- **Solution Idea:** (1) First element of preorder is the root. (2) Find root in inorder to split it into left/right subarrays. (3) Use lengths of these subarrays to split preorder. (4) Recurse.
- **Key:** Use a hash map for inorder indices to get $O(1)$ lookup.
- **Complexity:** $O(n)$ time, $O(n)$ space.

Problem: Binary Tree Maximum Path Sum (Hard, Blind 75)

- **Problem:** Find the maximum sum of any path in a binary tree.
- **Solution Idea:** Use a recursive DFS max_gain(node). It returns the max path sum *extendable upwards*. It also updates a global max_sum with the max path sum *contained at this node* (node.key + left_gain + right_gain).
- **Complexity:** $O(n)$ time, $O(h)$ space.

Problem: AVL Tree Insertion (from GFG Advanced-Data-Structure.pdf)

- **Problem:** Insert a node into an AVL tree and rebalance (Lec 27-29).
- **Solution Idea:** (1) Perform standard BST insert. (2) Walk back up from the new node to the root. (3) At each ancestor z, update its height. (4) Find the **first** ancestor z that violates the AVL property ($|h\_L - h\_R| > 1$). (5) Perform the correct **Rotation** (LL, RR, LR, RL) at z.
- **Key:** Only **one** rotation (single or double) is needed to fix an insertion.
- **Complexity:** $O(\log n)$ time.

Problem: B-Tree Insertion (from GFG Advanced-Data-Structure.pdf)

- **Problem:** Insert a key into a B-Tree (Lec 32).
- **Solution Idea (Pre-emptive Split):** We *never* insert into a full node.
  1. Start at the root. As we traverse *down* to find the correct leaf:
  2. **If we encounter a full node x:** We **split x first** (promoting its median key to its parent) before moving down into the correct (now non-full) child.
  3. **Finally:** We reach the target leaf, which is *guaranteed* not to be full. Insert key k.
- **Complexity:** $O(\log\_tn)$ disk accesses.

Problem: Implement Trie (Prefix Tree) (Blind 75)

- **Problem:** Implement a Trie with insert, search, and startsWith.
- **Solution Idea:** Use a TrieNode class with children[26] and isEndOfWord.
- **Complexity:** All ops are $O(L)$ time, where L is length of the word.

Problem: Auto-complete feature using Trie (from GFG Advanced-Data-Structure.pdf)

- **Problem:** Given a query string s, return all words in the dictionary that have s as a prefix.

- **Solution Idea:** (1) search the Trie for the node corresponding to the prefix s. (2) If this node p exists, run a DFS from p to find all words in the subtree rooted at p.
- **Complexity:** $O(L + K \cdot M)$ where L=length of prefix, K=number of suggestions, M=avg word length.

Module 7: Graph Algorithms

Problem: Number of Islands (Blind 75)

- **Problem:** Given a 2D grid of '1's (land) and '0's (water), count the islands.
- **Solution Idea:** Iterate the grid. If grid[r][c] == '1' and unvisited: increment island_count, and start a **BFS or DFS** to visit and mark all connected '1's.
- **Complexity:** $O(N \cdot M)$ time.

Problem: Topological Sort (from GFG Graphs.pdf)

- **Problem:** Find a linear ordering of vertices in a DAG such that for every edge (u, v), u comes before v (Lec 38).
- **Solution Idea (DFS):** (1) Run DFS on the graph. (2) As each vertex **finishes** (is marked BLACK), insert it onto the **front** of a linked list. (3) This list is the topological sort.
- **Correctness:** For an edge (u, v), v must finish before u (otherwise it's a back edge/cycle). Since u finishes later, it gets added to the front of the list, placing it before v.
- **Complexity:** $O(n + m)$ time.

Problem: Course Schedule (Lec 38, Blind 75)

- **Problem:** Can you finish all courses given prerequisites?
- **Solution Idea:** This is **Cycle Detection in a Directed Graph**.
- **Algorithm (DFS):** Use 3 "colors": WHITE (unvisited), GRAY (in current recursion stack), BLACK (finished). If DFS visits a GRAY node, a **back edge** (cycle) exists.
- **Complexity:** $O(n + m)$ time.

Problem: Detect Cycle in Undirected Graph (from GFG Graphs.pdf)

- **Problem:** Check if an undirected graph has a cycle.
- **Solution Idea 1 (DFS):** Run DFS. Keep track of each node's parent. If you find an edge to an already-visited node v that is **not** your parent, you have found a back edge/cycle.
- **Solution Idea 2 (DSU):** (Lec 41) Iterate all edges (u, v). For each edge, if Find(u) == Find(v): a cycle exists (they are already connected). else: Union(u, v).
- **Complexity:** $O(n + m)$ for DFS. $O(m \cdot \alpha(n))$ for DSU.

Problem: Snake and Ladder (from GFG Graphs.pdf)

- **Problem:** Find the minimum number of dice throws to win a Snake and Ladder game.
- **Solution Idea:** This is a **Shortest Path in an Unweighted Graph** problem.
- **Graph:** The board (squares 1-100) is the set of n vertices.
- **Edges:** From each square i, there are 6 edges to i+1, ..., i+6 (dice rolls).
- **Snakes/Ladders:** These are not edges, but "teleports". If a ladder is at i to j, you only have an edge to j, not i. (Model this in the adj. list).
- **Algorithm:** Run **BFS** from square 1. The dist[100] is the answer.
- **Complexity:** $O(n)$ time (where $n = 100$).

Problem: Word Search (Backtracking/DFS)

- **Problem:** Find if a word exists in a 2D grid. Cannot reuse a cell.
- **Solution Idea: DFS with backtracking**. Start DFS from any cell matching word[0]. Mark cell as visited, explore 4 neighbors, then un-mark cell.
- **Complexity:** $O(N \cdot M \cdot 3^L)$ where L is word length.

Problem: Kruskal's Algorithm for MST (from GFG Graphs.pdf)

- **Problem:** Find a Minimum Spanning Tree (Lec 41).
- **Solution Idea:** (1) **Sort** all m edges by weight ($O(m \log m)$). (2) Initialize a **DSU** with n sets. (3) Iterate sorted edges (u, v): if Find(u) != Find(v): add (u, v) to MST and Union(u, v).
- **Complexity:** $O(m \log m)$ or $O(m \log n)$ (dominated by sort).

Module 9: Exercise & Challenge Solutions

This module contains the solutions to the exercises and challenges presented in the lecture slides.

Module 2: Stacks & Queues

Problem: Efficient Array-Based Queue (Lec 8, Slide 54)

- **Problem:** Provide an efficient array-based implementation of a queue that requires $O(n)$ space (where n is max size) but n is not known beforehand.
- **Solution Idea:** Combine two techniques from the lectures.
  1. **Circular Array (Lec 8):** Use 'front' and 'back' indices with the modulo operator ('% N', where N is array capacity). This makes 'Dequeue' an $O(1)$ operation by just moving the 'front' pointer, avoiding an $O(N)$ shift.
  2. **Doubling/Halving (Lec 9):** To handle unknown size, start with a small array.
     - **On 'Enqueue':** If the array is full, create a new array of double the size ($2N$) and copy the N elements over. This gives an amortized $O(1)$ cost.
     - **On 'Dequeue':** If the number of elements falls below $N/4$, create a new array of half the size ($N/2$) and copy the elements. This prevents "thrashing" and maintains the amortized $O(1)$ cost.
- **Result:** All operations ('Enqueue', 'Dequeue') achieve $\mathbf{O(1)}$ amortized time complexity.

Module 3: Trees & Traversals

Problem: Traversal via Stacks and Queues (Lec 14, Slide 31)

- **Level-Order (using a Queue):** This is the BFS algorithm.
  1. Create a Queue, 'q.enqueue(root)'.
  2. While q is not empty: $u = q.dequeue()$.
  3. Visit u.
  4. If 'u.left != NULL', 'q.enqueue(u.left)'.
  5. If 'u.right != NULL', 'q.enqueue(u.right)'.
- **Preorder (using a Stack):**
  1. Create a Stack, 's.push(root)'.
  2. While s is not empty: $u = s.pop()$.
  3. Visit u.
  4. If 'u.right != NULL', 's.push(u.right)'. (Push right first)
  5. If 'u.left != NULL', 's.push(u.left)'. (Push left last)
- **Inorder (using a Stack):** This is the most complex iterative traversal.
  1. Create a Stack s. 'curr = root'.
  2. While 'curr != NULL' or s is not empty:
  3. **Go left:** While 'curr != NULL': 's.push(curr)', 'curr = curr.left'.
  4. **Visit:** 'curr = s.pop()'. Visit 'curr'.
  5. **Go right:** 'curr = curr.right'.

Problem: Count Trees from Level-Order (Lec 14, Slide 32)

- **Challenge:** "How many binary trees have the sequence ABCDEFG as their level-order traversal?"
- **Solution:** The level-order traversal sequence does *not* uniquely define a binary tree, because it does not encode 'NULL' children.
- **Example 1:** A *full, complete* tree: 'A.left=B', 'A.right=C', 'B.left=D', 'B.right=E', 'C.left=F', 'C.right=G'. This produces 'A B C D E F G'.
- **Example 2:** A "path" tree: 'A.right=B', 'B.right=C', 'C.right=D', 'D.right=E', 'E.right=F', 'F.right=G'. This also produces 'A B C D E F G'.
- **Conclusion:** There are many possible trees. However, as noted on Lec 14, Slide 47, if the tree is constrained to be an **Almost Complete Binary Tree**, the sequence uniquely defines **exactly one** tree.

Module 6: BSTs & AVL Trees

Problem: Inorder Traversal of BST is Sorted (Lec 25, Slide 6)

- **Proof (by Induction):**
- **Base Case:** $n = 0$ (empty tree) or $n = 1$ (root only). The traversal is empty or has one node, which is sorted.
- **Inductive Hypothesis (IH):** Assume that for any BST with $k < n$ nodes, an inorder traversal produces a sorted list.
- **Inductive Step (for n nodes):** Let T be a BST with n nodes and root r. T's left subtree T_L and right subtree T_R are both BSTs with $< n$ nodes.
  1. By the **BST Invariant** (Lec 24), all keys in T_L are $\leq r$.key, and all keys in T_R are $\geq r$.key.
  2. An inorder traversal of T is: 'Inorder(T_L)', then 'r.key', then 'Inorder(T_R)'.
  3. By the (IH), 'Inorder(T_L)' is a sorted list.
  4. By the (IH), 'Inorder(T_R)' is a sorted list.
  5. Combining these facts, the full traversal is: [sorted list $\leq r$] + [r] + [sorted list $\geq r$].
  6. This final combined list is fully sorted.

Problem: Find All Occurrences of Key k (Lec 25, Slide 17)

- **Problem:** Design an algorithm to find all occurrences of k.
- **Solution Idea:** The standard 'Search' stops at the first k. Since duplicates $\leq k$ can be in the left subtree and $\geq k$ can be in the right, we must search both.
- **Algorithm** 'FindAll(node, k, results_list)':
  1. If 'node == NULL', return.
  2. If 'k ¡ node.key':
  3. 'FindAll(node.left, k, results_list)'
  4. Else if 'k ¿ node.key':
  5. 'FindAll(node.right, k, results_list)'
  6. Else ('k == node.key'):
  7. 'results_list.add(node)'
  8. 'FindAll(node.left, k, results_list)' (to find other k's in left)
  9. 'FindAll(node.right, k, results_list)' (to find other k's in right)
- **Complexity:** $O(h + c)$, where h is tree height and c is the number of occurrences.

Problem: Compute Predecessor (Lec 25, Slide 32)

- **Problem:** Modify the 'Successor' algorithm to compute 'Predecessor(x)'.
- **Solution Idea:** This is the symmetric (mirror-image) of 'Successor'.
- **Algorithm:**
  1. **Case 1: If x has a left subtree.** The predecessor is the `Maximum()` node in x's left subtree.
  2. **Case 2: If x has NO left subtree.** Go up the tree using 'parent' pointers. The predecessor is the **first ancestor** y such that x is in the **right subtree** of y.
- **Complexity:** $O(h)$.

Problem: Expected Height of Random BST (Lec 25, Slide 52)

- **Problem:** What is the expected height of a BST constructed from a random permutation of $\{1, \ldots, n\}$?
- **Solution (from CLRS 12.4):**
- **Result:** The expected height is $\mathbf{O}(\log \mathbf{n})$.
- **Proof Sketch:** Let X_n be the height of a random BST on n nodes. Let R be the rank of the key chosen as the root (with $P(R = i) = 1/n$).
- The height of the tree is $1+\max(X\_i - 1, X\_n - i)$, where $X\_i - 1$ and $X\_n - i$ are the heights of the left/right subtrees.
- The analysis of this recurrence (which is very similar to the analysis of randomized quicksort) shows that $E[X\_n] = O(\log n)$.

Problem: Maintain Subtree Size in BST (Lec 26, Slide 5)

- **Problem:** Update 'Insert' and 'Delete' to maintain the 'size' field at every node.
- **Solution ('size(x) = size(x.left) + size(x.right) + 1'):**

- **Insert(z):**
  1. Perform a normal BST insert. ‘size(z)‘ is initialized to 1.
  2. As the search path is traversed *down* to find the insertion spot, increment the ‘size‘ of every node on that path.
- **Delete(z):**
  1. Perform a normal BST delete. Let y be the node that was physically removed (either z or z’s successor).
  2. Starting from ‘parent(y)‘, walk *up* the tree to the root, decrementing the ‘size‘ of every node on that path.
- **Complexity:** Both operations remain $O(h)$.

Problem: AVL Rotation Height Proof (Lec 28, Slides 42, 47)

- **Problem:** In Case 1(a) (Right-Rotate) and Case 2(a) (Left-Rotate), prove the middle subtree (yellow box) must have height $h - 3$.
- **Proof (for Case 1a / Right-Rotate):**
  1. Let Y be the unbalanced node. Its height *after* insertion is h.
  2. Let X be its left child. Since this is Case 1a, X is left-heavy and Y is unbalanced.
  3. This means ‘h(X) = h-1‘ and ‘h(Y.right) = h-3‘.
  4. The imbalance was caused by an insertion into X’s left subtree, so X’s height just grew from $h - 2$ to $h - 1$.
  5. Since Y *was* balanced before the insert (when ‘h(X) = h-2‘), its other child ‘Y.right‘ must have had height $h - 3$ (as $h - 2$ would also be balanced).
  6. Since X *was* balanced before the insert (at height $h - 2$), and the insert occurred in its *left* child (making it $h - 2$), its *right* child (the yellow box) must have had height $\leq h - 3$.
  7. To maintain X’s balance *before* the insert, if $X.left$ was $h - 3$, $X.right$ must have been $h - 3$.
  8. Thus, the yellow box’s height *before* insertion was $h - 3$, and it was unchanged by the insertion.